

18662 Project Final Report

LLM-Powered Distributed Agents for Cooperative Planning

Yixiang Wang, Xianfei Yang, and Haoran Zhang

Abstract—This study explores the application of large language models (LLMs) as decentralized decision-makers in collaborative multi-agent systems, harnessing their pre-trained knowledge for dynamic task coordination. We start from the existing framework (Mcrafter), highlighting its inefficiency and limitations. Then we introduce a novel, low-cost game framework that enables detailed observation of inter-agent interaction and cooperation. Our framework leverages OpenAI’s structured input-output interfaces to process game rules and interaction history. Through experiments within this framework, we evaluate methods for enhancing agent performance, identify limitations related to memory input in specific scenarios, and analyze the impact of multilingual prompts. This work provides a critical assessment of LLMs’ capabilities and constraints as real-time decision engines in open multi-agent environments.

Declaration:

Given that the team has substantially modified the project direction by developing a new simulation environment from scratch, we concluded after discussion that presenting the final work as a direct extension of the mid-term report would not be appropriate. Such an approach would likely introduce significant inconsistencies and redundancies, potentially leading to confusion for our readers. Therefore, we have decided to reorganize and present the final report as follows:

- 1) Introduction
- 2) Related work
- 3) Problem Formulation and Problem Statement
- 4) Old Approach & Preliminary Results
- 5) New Approach & Framework
- 6) Experiment Results & Discussion
- 7) Final Thoughts & Future works
- 8) Conclusion

I. INTRODUCTION

Decentralized multi-agent systems (DMAS) [1] have long been a central research area in artificial intelligence, with applications in robotics [2], autonomous vehicles [3], health-care [4], and collaborative problem-solving [5]. Recent advancements in large language models (LLMs) have introduced new opportunities for decentralized multi-agent collaboration, enabling agents to reason, plan, and communicate in complex environments [6]. However, many existing approaches, such as centralized training with decentralized execution (CTDE) in Multi-Agent Reinforcement Learning (MAREL) [7], [8], struggle with scalability and flexibility. Specifically, they assume either fixed cooperation strategies or require custom reward functions, limiting their adaptability to dynamic, open-ended tasks.

Recent work has explored LLM-powered generative agents in decentralized settings, introducing adaptive hierarchical knowledge graph structures and structured communication protocols to facilitate long-term planning and collaboration [9]. The framework, applied to open-world environments like Crafter [10], demonstrated significant performance improvements over traditional MARL approaches by leveraging structured memory and adaptive knowledge representation. However, a significant limitation arises from the framework’s exclusive use of game screenshots as input. This not only introduces unnecessary complexity by requiring precise processing of rich visual data but also necessitates the use of highly powerful LLMs, posing a barrier for many researchers.

Research Challenges While existing work, such as [9], has demonstrated the application of LLM agents for cooperative task completion in complex environments, several challenges remain in advancing agent design and understanding LLM interactions. Our research aims to address the following key areas:

- **Developing a Low-Cost and Efficient Framework:** Current frameworks, such as the one used in Mcrafter [9], often rely on high-dimensional inputs like game screenshots for agent decision-making. This approach introduces unnecessary complexity and necessitates the use of extremely powerful, and thus computationally expensive, models to observe clear cooperative behaviors. A significant challenge is to develop a more efficient and lower-cost framework that can facilitate cooperative interactions even when utilizing models with lower capacities.
- **Enhancing Agent Human-Likeness and Cognitive Processes:** Prior work mainly employs summarized planning mechanisms for agents. However, for distributed agent systems, individual planning strategies are often more suitable and could lead to more flexible behaviors. Furthermore, the precise impact of memory processes on agent behavior and decision-making within cooperative or individual tasks remains an area requiring deeper exploration. Understanding and effectively modeling these cognitive aspects are crucial for developing more human-like agents.
- **Investigating the Impact of Multilingual Prompts:** The majority of training data for prominent LLMs, particularly those from OpenAI, is English-centric. Consequently, the influence of prompts written in languages

other than English on the performance, behavior, and cooperative abilities of these agents is not well understood. Research into the effects of multilingual prompts is essential for developing globally applicable and robust LLM agents.

Contributions: Addressing the research challenges outlined in the preceding section, we first provide a review of related work in Section II and then present the following key contributions:

- 1) **A Low-Cost and Simplified Framework for Decentralized Agent Cooperation:** We introduce *Mcraftmini*, a novel, simplified resource collection game framework specifically designed to facilitate the deployment and experimentation of even "tiny" LLMs in decentralized cooperative settings. This framework directly addresses the first challenge by providing an efficient and accessible platform for studying agent interactions without the high computational overhead of existing systems.
- 2) **Empirical Analysis and Guidelines for Human-Like LLM Agents:** We conduct extensive empirical studies investigating various aspects of LLM agent performance critical for human-like behavior. Our experiments explore different planning granularities (low-level versus high-level), planning strategies (summarized versus individual), and the influence of memory capabilities (rich versus poor) across diverse model scales (e.g., utilizing models such as GPT-4o and a hypothetical smaller model like GPT-4.1-nano). The results of these experiments are synthesized into a set of critical guidelines for the design and implementation of more human-like and effective LLM agents, thereby tackling the second challenge.
- 3) **Insights and Improvements Regarding Multilingual Prompting:** We systematically examine the performance of LLM agent systems when prompted in different languages. Our findings reveal that prompts in English, despite being the primary training language for many models, can occasionally lead to suboptimal performance in certain configurations. We identify specific instances where this occurs and propose improvements for crafting English prompts to enhance agent performance and reliability, offering a practical solution to the third challenge concerning multilingual impact.

II. RELATED WORK

Multi-Agent Cooperation and Communication. Multi-agent reinforcement learning (MARL) has become instrumental in advancing decision-making in tasks reliant on communication [8]. Various approaches have emerged, categorized primarily into continuous and discrete communication methods. Continuous communication approaches allow for nuanced information exchange but can result in high communication overhead and reduced interpretability [11]. In contrast, discrete communication methods minimize overhead but may sacrifice expressiveness and relational learning capabilities [12]. Although techniques like learnable messages [13] attempt to

bridge this gap, they often require extensive vocabularies and lack guarantees in decentralized partially observable Markov decision processes. Recent advances have utilized large language models (LLMs) to enhance multi-agent communication and planning in dynamic environments. Frameworks such as CoELA [14] employ free-form natural language generation, yet they lack structured planning and reasoning about the consequences of actions. Other works, such as SPRING [15], integrate external knowledge for planning but do not sufficiently utilize in-environment interactions for learning.

Generative Agents and Structured Reasoning. The use of LLMs for planning and decision-making tasks has rapidly expanded, highlighting their potential for real-time agent guidance in interactive environments [16], [17]. However, challenges persist, especially when handling complex, multi-objective tasks in expansive environments, which demand structured reasoning and hierarchical task decomposition [18]. Recent efforts include structured prompting techniques and converting unstructured inputs into structured outputs [19], thereby improving agent workflows and decision-making accuracy.

Adaptive Knowledge Graph Memory Systems. Recent literature emphasizes the importance of integrating adaptive knowledge graph memory systems (A-KGMS) to enhance agent cooperation and planning capabilities [20], [21]. These knowledge graphs maintain goal-oriented records of agent experiences, facilitating efficient retrieval and adaptation based on past events and ongoing tasks. Such structured memory systems empower agents to dynamically balance long-term strategic planning with immediate, tactical decision-making, significantly outperforming traditional MARL approaches that lack memory systems.

III. PROBLEM FORMULATION AND PROBLEM STATEMENT

In both real-world and virtual environments, tasks can be executed either individually or through multi-agent collaboration. When a task is performed by a single agent in a fully observable environment with well-defined prior probabilities, established methods such as Bayesian Networks can be employed to systematically infer the optimal solution path. These methods leverage probabilistic reasoning to model dependencies and compute the most efficient course of action based on available data. However, as the number of agents increases or when the environment becomes partially observable, explicitly constructing a complete decision model becomes computationally infeasible due to the exponential growth of state-action space and the inherent uncertainty in inter-agent interactions. Under such conditions, alternative approaches must be developed to facilitate efficient task coordination and decision-making.

LLMs offer a promising solution in this context, as they encapsulate vast amounts of prior knowledge and can generate adaptive strategies with minimal computational overhead compared to traditional model-based planning techniques. By leveraging pretrained representations of complex relationships and task structures, LLMs can guide multi-agent collaboration in environments with incomplete information. However, a

fundamental limitation of LLMs is their lack of persistent memory, meaning they do not retain past experiences across decision cycles. Consequently, each time the environment is reset or a new episode begins, the model reconstructs its decision-making process from scratch, failing to incorporate prior learning to optimize future iterations. This absence of memory retention leads to redundant computations, inefficient task execution, and suboptimal policy adaptation over time.

To address this limitation, external memory mechanisms are essential for storing historical information and enabling agents to retrieve and reflect on past experiences. By integrating structured memory retrieval techniques, agents can dynamically refine their decision-making processes, leveraging prior knowledge to improve task efficiency and reduce computational redundancy. Incorporating long-term memory structures further enhances multi-agent collaboration by allowing agents to accumulate and utilize contextual insights over time, thereby optimizing sequential decision-making and improving adaptability in dynamic environments.

IV. OLD APPROACH & PRELIMINARY RESULTS

Here, we briefly describe what we got after trying Hanqing’s code. The previous framework, MCRAFTER [9], aimed to utilize LLMs as decentralized decision-makers for collaborative multi-agent systems in a simulated open-world environment inspired by Crafter. The framework integrated OpenAI’s structured input-output interfaces, processing three primary inputs for agent decision-making: game rules, real-time agent observations (provided as pixel-based PNG images), and historical interaction logs. Agents operated autonomously under customized rules, employing structured communication protocols and decentralized knowledge-sharing mechanisms, including an Adaptive Knowledge Graph Memory System (A-KGMS) to retain historical information and facilitate adaptive planning. The intention was to leverage the pre-trained knowledge of LLMs for dynamic task coordination and overcome limitations of traditional Multi-Agent Reinforcement Learning (MARL) approaches, such as scalability issues and the need for fixed strategies or custom reward functions.

However, our preliminary experiments with this framework, as detailed in the mid-term report, revealed significant limitations and challenges that motivated the development of our new approach. The key problems encountered were:

- **Ineffectiveness of Image Recognition:** A critical limitation stemmed from the framework’s reliance on LLMs processing pixel-based PNG images as real-time agent observations. Due to the low resolution of the images and the complexity of the environment, the LLM-driven image recognition module frequently produced inaccurate interpretations of the visual environment. Examples included misclassifying health status or mistaking environmental elements like grass for agents (e.g., “cow”). These misinterpretations led to erroneous decision-making and suboptimal agent actions, often resulting in system failures or agents entering unproductive loops.
- **Poor Scalability:** Despite simplifying game rules significantly and augmenting LLM prompt token capacity

(which increased cost), successful gameplay was only achievable with a maximum of two agents. The system failed to scale effectively beyond this, indicating fundamental scalability limitations inherent in the approach, particularly concerning the complexity introduced by image processing and the coordination overhead with more agents.

- **High Experiment Cost:** The necessity of using powerful models (like GPT-4o mentioned in the report) capable of processing complex inputs like images, combined with the need to increase input tokens for even simplified tasks, resulted in a high computational and financial cost for conducting experiments. This made extensive exploration and testing, especially with lower-capacity models, impractical within this framework.
- **Sensitivity to Prompt Structure:** While prompt engineering and the knowledge graph partially mitigated reasoning errors, the LLM’s decision-making was highly sensitive to prompt structure and the explicit inclusion of information. Unstructured prompts led to disorganized reasoning and a failure to recognize task dependencies, while explicitly including certain information (like health status) could lead agents to prioritize irrelevant goals, hindering task efficiency. This suggested a lack of robust intrinsic reasoning independent of prompt design.

These preliminary results highlighted that while the foundational idea of using LLMs for decentralized multi-agent cooperation is promising, the specific implementation choices in MCRAFTER, particularly the reliance on image inputs and the resulting scalability and cost issues, presented significant barriers to further research and experimentation, especially with smaller models. This necessitated a fundamental re-assessment and redesign of the framework to create a more efficient, low-cost, and scalable platform for studying LLM agent cooperation.

V. NEW APPROACH & FRAMEWORK

A. Environment and framework Builds up

Considering the poor performance and unresolvable blocking issues the team encountered when using the old frame, the team decided to build a simplified framework from scratch (Please refer to preliminary results).

1) *Inspiration:* Inspired by open-world environments like MineDojo [22] and Crafter [10], our objective was to build a new environment that is capable of not only benchmarking but also testing LLM-based decision-making systems in resource collection tasks. In our initial design, we developed **2 progressively complex multi-agent environments**: Simplified and Complex. Each environment is generated procedurally with randomized resources and supports integration with structured LLM-based planning. The specific design will be thoroughly discussed in the following section.

2) *Design:* To facilitate more reliable and interpretable interaction between language models and autonomous agents, we designed and implemented a custom multi-agent simulation environment based on a structured 2D grid. Unlike previous approaches that completely relied on the image representation

of the maps, our framework adopts a simplified but functionally complete environment, allowing for clearer reasoning and efficient evaluation of LLM-based planning.

The environment consists of a discrete 2D grid of fixed dimensions, where each cell represents a possible location for agents or resources. At the start of each episode, resources—including wood, stone, iron, and diamond—are randomly distributed across the grid, ensuring variation and generalization across runs. Each agent is initialized at a distinct location (in our example, left-up and right-bottom), as 1 shows. Most importantly, agent is granted full observability of the global map state. This means that agents have complete knowledge of their own positions as well as the coordinates and types of all resource locations at each step.



Fig. 1: Simplified game environment

To ensure a clearer and more observable task execution, we followed depth-first principles to simplify the rules. The core part of the rule is a strict but simple collection sequence: agents must first collect wood, followed by stone, then iron, and finally diamond. Unlike traditional crafting environments that require intermediate tool-building or resource processing such as building table, axes, and furnace, we eliminated such intermediate procedures to reduce task complexity and focus on evaluating planning and collaboration capabilities.

Crucially, we represent the environment using a matrix-based map representation instead of visual or image-based inputs (Figure 2). Each cell in the matrix is labeled with an integer corresponding to its contents: e.g., -1 for agent 1, -2 for agent 2, 1 for wood, 2 for stone, etc (Figure 3). This design choice allows the LLM to reason over a structured, symbolic input rather than needing to infer spatial layout from images or screenshots. As a result, the system is significantly less prone to hallucination, misinterpretation, or ambiguity commonly encountered when dealing with unstructured visual inputs.

By using this approach, we not only reduce the computational and interpretive burden placed on the LLM, but also create a testbed that enables transparent evaluation of agent behavior, collaboration patterns, and planning decisions under well-defined and reproducible conditions.



Fig. 2: Image representation of map

$$\begin{bmatrix} -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 2 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 3 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 4 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 2 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 3 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 4 & 3 & 2 & 4 & 1 & -2 \end{bmatrix}$$

Fig. 3: Matrix representation of map

B. Integration of LLM into decision making

In this framework, a large language model (LLM) is employed as a centralized planner to guide agent decision-making in a resource collection task. The integration between the simulation environment and the LLM is carefully designed to ensure both interpretability and actionability. At each time step, the environment constructs a structured input that is passed to the LLM, and the LLM responds with structured output that directly informs the next action of one or more agents.

LLM Input Design

To support effective planning, the LLM receives the following components as input:

- 1) **Global Game Rules:** A concise description of the task rules is included in the prompt, specifying the resource collection order (wood → stone → iron → diamond), the shared nature of collected resources, and the assumption of global observability.
- 2) **Agent Information:** The input includes the position and identity of each agent, as well as the agent for whom the current query is being made (in the case of individual planning).
- 3) **Memory Context:** A short memory buffer is appended to the prompt, recording the most recent actions, goals, and outcomes for both agents. This provides the LLM with temporal context, enabling it to make decisions that are informed by recent history and coordination patterns.
- 4) **Map Matrix:** Described in the previous section.

Structured Output as a Control Interface:

To ensure consistency, safety, and ease of execution, the LLM is required to respond in a structured output format, typically a JSON object conforming to a predefined schema. Each output includes:

- **agent id:** specifies which agent the action is for.
- **action:** one of move, wait.
- **direction:** one of up, down, left, right
- **target:** the [x, y] coordinate of the resource being approached
- **goal:** the resource type (e.g., wood, stone, etc.)
- **reason:** a natural language justification of the decision

Figure 4 shows the actual implementation of structured output using Python, and Figure 5 shows an example of output during the game simulation.

```
class AgentAction(BaseModel):
    agent_id: Literal["agent_1", "agent_2"]
    action: Literal["move", "wait"]
    direction: Optional[Literal["up", "down", "left", "right"]] = None
    target: Optional[List[int]] = None
    reason: str

class AgentPlan(BaseModel):
    actions: list[AgentAction]

def ask_gpt_to_plan(env, warehouse_summary):
    planner_input = build_planner_input(env, warehouse_summary)
    system_prompt = build_prompt()

    # print(f"planner input is: {planner_input}")
    # print(f"system prompt is: {system_prompt}")

    response = client.beta.chat.completions.parse(
        model="gpt-4o-2024-08-06", # or gpt-4o-2024-08-06
        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": json.dumps(planner_input)}
        ],
        response_format=AgentPlan,
    )
    return response.choices[0].message.parsed
```

Fig. 4: An implementation example of structured output in Python

```
{
  "agent_id": "agent_1",
  "action": "move",
  "direction": "right",
  "target": [0, 1],
  "reason": "moving toward nearest wood"
},
{
  "agent_id": "agent_2",
  "action": "move",
  "direction": "up",
  "target": [13, 14],
  "reason": "moving toward nearest wood"
}
```

Fig. 5: An example of structured output

This structured output serves as a powerful interface between natural language reasoning and environment-level control. It offers several key advantages:

- **Validity and Executability:**

By constraining the output format, we ensure that all generated actions are valid and directly mappable to in-game operations.

- **Interpretability and Debuggability:**

The inclusion of both symbolic actions and natural language rationales enables easy debugging and provides insights into the model's reasoning process (Chain of thoughts).

- **Modularity and Scalability:**

Structured output makes it possible to extend the system to more complex actions (e.g., collect, build) in the future without changing the communication protocol.

- **Reduced Ambiguity:**

Compared to free-form natural language commands, structured outputs eliminate ambiguity in the agent's next move and make the decision pipeline more robust.

Overall, the integration of structured inputs and outputs ensures that the LLM operates not as a black-box generator, but as a constrained, transparent, and interpretable planner within a closed-loop decision-making system.

C. New planning strategies

The two main planning strategies used for guiding multi-agent resource collection are **Movement-Based** and **Target-Based** (Figure 6). Both approaches share a common input format—structured representations of the map, game rules, agent positions, and recent memory context—but differ fundamentally in the method with which the LLM guides agents.

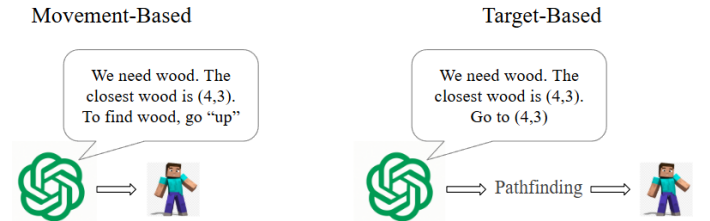


Fig. 6: Movement-Based vs Target-Based

Movement-Based Planning:

In the movement-based strategy, the LLM is responsible for providing fine-grained, step-by-step movement instructions to the agents. At each decision point, the LLM outputs a direction (up, down, left, or right) that the agent should move in for the next time step. This strategy tightly couples the LLM to the agents' immediate navigation decisions, requiring it to reason not only about high-level goals but also about short-term movement sequences. Here are some advantages and disadvantages:

Advantages:

- Provides detailed guidance at every step.
- Enables flexible adaptation to dynamic environmental changes.

Disadvantages:

- Higher token consumption due to frequent queries and fine-grained instructions.
- Increased susceptibility to cumulative planning errors over long trajectories.
- Potentially slower overall decision-making due to LLM communication overhead

Target-Based Planning:

In contrast, the target-based strategy requires LLM to conduct high-level planning. Instead of specifying immediate moves, the LLM outputs a target coordinate $([x, y])$ representing the location of a resource that the agent should navigate toward. The low-level path finding to reach the target is handled locally by the environment using a classical A*-based algorithm, which computes an optimal path avoiding obstacles and minimizing travel cost. Here are the advantages and disadvantages of this approach:

Advantages:

- Significantly reduces token consumption, since fewer instructions are needed per task.
- Separates high-level strategic reasoning (handled by the LLM) from low-level motion planning which is handled locally. (later experiments show that LLM is much better at performing high-level planning)
- Improves planning robustness by leveraging proven deterministic algorithms for navigation.

Disadvantages:

- Requires that the environment provides reliable local pathfinding support.
- Less responsive to dynamic changes if targets become outdated between planning steps.

Both planning strategies use the same structured input. Movement-based planning tends to incur higher token consumption and computation costs due to its step-by-step mechanism, while target-based planning offers a more efficient and scalable alternative by decoupling goal selection from path execution. The detailed description of each approach will be presented in the following result section.

D. Memory-Augmented Planning

To improve temporal coherence and multi-agent coordination, we implemented a memory-augmented planning system, in which the LLM leverages both short-term and long-term memory to inform its decisions. The memory mechanism is visually illustrated in the Figure 7.

At each decision-making step, the agents query the LLM with the question “What’s our next step?” The LLM receives not only a structured input but also the recent memory, which contains the last few actions, goals, and positions of both agents. In our case, we used the last 5 segments of total memory as recent memory.

The LLM processes this information and outputs a plan in a structured format. The result is then stored into the Total Memory, which serves as a cumulative, chronological record of all actions taken throughout the episode—such as “Agent 1 collected wood at step 12”. This total memory is used not

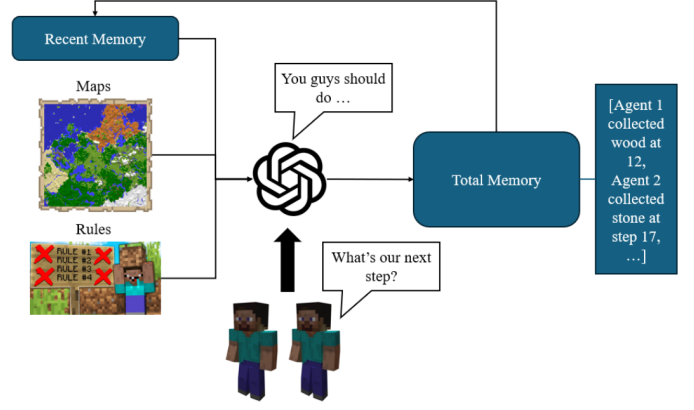


Fig. 7: Memory Mechanism

only for post-analysis but also can be partially summarized and passed back to the LLM as contextual input. More importantly, adaptive knowledge graph, even though not implemented in our project, is built from memory.

Theoretically, the memory should be useful as it plays three essential roles:

- 1) **Contextual enhancement:** Recent memory helps the LLM maintain local coherence and continuity of planning.
- 2) **Collaborative Reasoning:** By referencing past actions of both agents, the LLM can infer whether redundant actions are being repeated and adjust plans accordingly.
- 3) **Interpretability and Logging:** The total memory serves as a transparent history of agent behavior, enabling both debugging and performance analysis.

By integrating memory into the interaction loop, the system transitions from single-step planning to history-attention decision-making.

E. Joint vs. Individual

To explore how large language models (LLMs) reason about multi-agent coordination, we implemented and compared two planning paradigms: joint planning and individual planning. The distinction between these approaches is conceptually similar to the difference between asking “What should I do?” versus “What should we do?”

In the **joint planning**, the environment submits a single consolidated prompt to the LLM on behalf of all agents. The prompt includes the global map, recent memory, and game rules, etc. In return, the LLM generates a batch of coordinated actions, one for each agent, in a shared reasoning context.

In contrast, the **individual planning** (Figure 8) treats each agent as an independent planner that queries the LLM separately. However, each agent’s prompt still includes the current plan and condition of the other agent, along with the same shared inputs—map matrix, recent memory, and game rules.

The goal of this design is to examine whether the LLM can exhibit collaborative behavior implicitly, even when agents query independently. Specifically, we seek to understand whether the LLM can reason, for instance, that “since Agent 1 is collecting wood, Agent 2 should begin moving toward

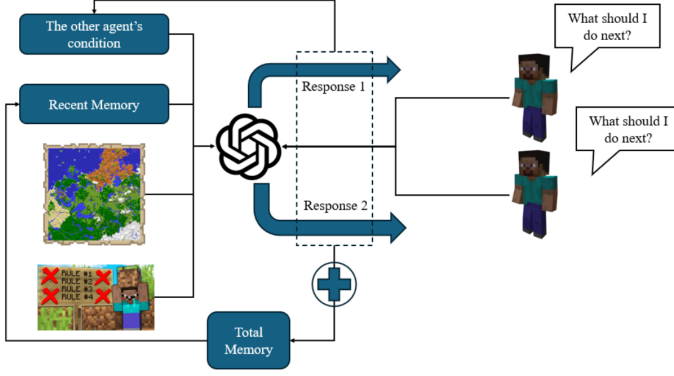


Fig. 8: Individual Planning

stone.” This tests the model’s ability to coordinate asynchronously, conditioned on shared memory and other-agent context.

F. Complication of Task

Building upon the basic setting of the previous environment, we introduce several modifications to increase the complexity and realism of the task. These changes are designed to better evaluate the reasoning, planning, and collaboration abilities of large language models (LLMs) in a more dynamic and challenging environment. Specifically, three major modifications are implemented:

- 1) **Increase in the number of agents:** The number of agents is increased from two to four. This adjustment is intended to create a more collaborative environment where agents must coordinate their actions more efficiently. With more agents present, task allocation and inter-agent communication become increasingly important for achieving the final objective, thus providing a more stringent test of the LLM’s planning and coordination abilities.
- 2) **Expansion of resource types and quantities:** Inspired by the resource model used in Minecraft, we introduce a new type of resource—coal—which plays a crucial role in the process of collecting diamonds, an advanced resource necessary for task completion. Additionally, we increase the quantities of each resource throughout the environment. This design choice aims to lower the likelihood of bottlenecks during resource collection, enabling smoother progression and allowing the agents to focus on higher-level planning and tool creation strategies rather than being constrained by basic survival needs.
- 3) **Introduction of a tool system and dependency structure:** To further challenge the decision-making and long-term planning capabilities of the LLM, we introduce a structured tool system with explicit dependencies among tools, as illustrated in Figure 10. Agents must craft specific tools to access higher-tier resources; for example, certain resources cannot be collected without first building prerequisite tools. The creation of new tools requires

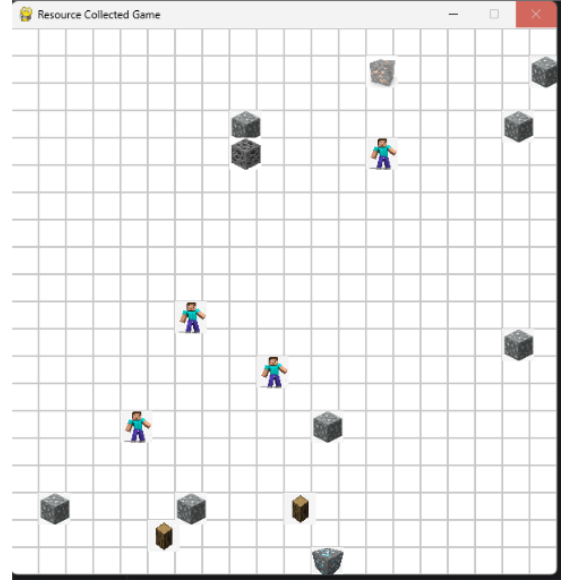


Fig. 9: Illustration of the more complicated environment.

agents to satisfy both resource and tool prerequisites, fostering more strategic behavior and dependency-aware planning. In the figure, an arrow from one tool to another indicates that the target tool requires the resources and tools associated with the source node. This additional layer of complexity forces agents to engage in multi-step reasoning, sequential planning, and effective resource management.

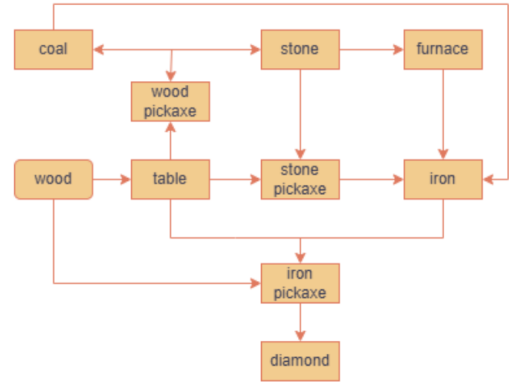


Fig. 10: Dependencies among tools.

VI. EXPERIMENT RESULTS & DISCUSSION

A. Experiment 1: movement-based with simplified environment

Our initial experiments start with movement-based planning strategies. To systematically evaluate the planning behavior of LLM in this setting, we designed four progressively more complex experimental setups. Each setup varies in terms of the information provided to the LLM, while all follow a joint or individual planning scheme and share the same matrix-based environment representation.

1) *Experiment setups*: Following is the summary of these 4 settings:

- **Normal Setup:**

The baseline configuration provides the LLM with the map matrix and a brief summary of the general game rules. The output includes a structured action for each agent (e.g., movement direction), along with the intended goal, target location, and a natural language reason. This setup follows joint planning, where both agents are planned simultaneously and no memory of previous steps is used.

- **Longer Prompt:** This setup extends the baseline by incorporating more detailed game rules into the LLM input prompt. For example, instead of saying "collecting order: wood → stone → iron → diamond", we specify it as "As long as the agent collect one wood, go ahead and collect stone, do not waste more time on collecting wood." Another example is that instead of saying "Please direct the agent to collect resources", we specify it as "Please make sure that your decision will direct the agent closer the target, nor further. Also select the nearest resources for the agent not further ones." The purpose is to examine whether providing a more comprehensive rule specification and hints leads to improved behavior or deeper rule understanding. Like the normal setup, this configuration still uses joint planning and does not include memory.

- **Memory Added:**

In this setup, we use the regular rule descriptions and additionally include the last 5 steps of memory for both agents. This "recent memory" contains prior actions, targets, and outcomes, providing the LLM with temporal context that may improve continuity and avoid redundant decisions. Planning is still joint, but now memory is incorporated into the prompt.

- **Individual Plan**

As explained before, in this setup, each agent queries the LLM separately. However, the prompt for each agent includes not only the map and memory, but also the current plan of the other agent. This setup is specifically designed to test whether the LLM can exhibit emergent collaborative behavior, despite being queried by agents independently.

2) *Qualitative results*: Following are the qualitative results (observation of agent behavior) for each setup. The results show a variety of behavioral patterns and limitations in how LLMs reason about spatial navigation and multi-agent coordination.

- **Normal Setup:**

In the baseline condition, agents frequently exhibited non-optimal and repetitive behaviors, such as rotating in place or getting stuck in corners, resulting in excessive and inefficient movement. Although Agent 1 may correctly approach a nearby wood resource, Agent 2 often redundantly attempts the same task, instead of pursuing a next-level resource such as stone. Additionally, agents occasionally made unreasonable movement. For example,

moving away from a nearby resource instead of toward it. This suggests that, under minimal input conditions, the LLM fails to accurately track proximity or conduct short-range spatial reasoning. Moreover, the LLM demonstrates limited rule understanding, continuing to collect multiple units of wood even after the prerequisite for collecting stone has been fulfilled.

- **Longer Prompt:**

Adding more detailed rule descriptions resulted in the worst performance among all experiments. Redundant task assignment persisted, and agent behavior became more erratic. In some instances, one agent followed a seemingly correct plan, while the other remained idle. The LLM also struggled to interpret map coordinates reliably, often producing imprecise or infeasible movement directions—such as instructing an agent in the bottom-right corner to move further down or right. Notably, this setup increased LLM inference latency due to longer input tokens, but did not yield better performance. While the model generated well-formed natural language rationales, they were frequently misaligned with the actual actions, indicating a disconnect between explanation and execution.

- **Memory Added:**

Incorporating recent memory led to notable improvements in efficiency, with agents completing tasks using fewer steps than in prior setups. Although occasional local rotation behavior remained, it was less frequent and less severe. However, the agents still lacked coordination: both frequently moved toward the same resource, showing limited emergent collaboration, despite having access to shared memory history.

- **Individual Planning:**

Surprisingly, the individual planning mode performed worse overall than joint planning. Agents tended to select suboptimal, distant resources, rather than the nearest valid ones. Nonetheless, there were occasional signs of implicit collaboration: for instance, when Agent 1 was collecting wood, Agent 2 moved toward a stone resource. This suggests that the LLM can reason about task distribution to a limited degree when conditioned on the other agent's plan. However, local rotation and stagnation remained, and the lack of centralized coordination may have contributed to the decreased overall efficiency.

3) *Quantative results*: For each setup, we conducted experiments across 100 independent game episodes.

Figure 11 illustrates the average number of steps take to complete a full game under four different setups. Error bars represent the standard deviation across multiple runs. Table I shows the exact statistical values of the results.

We observe that the "Longer and tedious prompt" setup yields the worst performance, requiring on average over 600 steps with highest variance. This indicates that simply increasing prompt length does not improve planning quality and, in fact, may degrade it—likely due to increased token complexity and cognitive overload for the LLM.

In contrast, the "Memory added" configuration demonstrates the most efficient performance, completing games in under 50



Fig. 11: Movement-based experiment results

steps on average with minimal variability. This highlights the critical role of memory in improving planning continuity and reducing redundant agent actions.

The "Normal setup" performs moderately well, while the "Individual Asking" setup incurs a slight performance drop compared to memory-augmented joint planning. This suggests that while providing agent-specific prompts enables some collaboration, the lack of joint optimization reduces overall efficiency.

These results reinforce the importance of temporal memory and prompt structure in optimizing LLM-driven multi-agent decision-making.

TABLE I: Steps taken to complete one game under different setups

	Normal Setup	Longer Prompt	Memory Added	Individual Planning
Mean	141	619	43	161
Variance	10	15	5	15

Based on the results of the movement-based experiments, we observe that requiring LLM to execute low-level, step-by-step action planning does not lead to effective collaboration between agents. Throughout all settings, instances of task redundancy persist, where multiple agents repeatedly pursue the same resource without coordinating their actions. More critically, when tasked with planning specific movement paths toward a given resource, the LLM consistently fails to generate optimal trajectories. Agents often take unnecessarily convoluted paths, suggesting that the LLM lacks sufficient spatial reasoning capability to plan efficient routes. Taken together, these findings highlight a fundamental limitation: LLMs are not well-suited for fine-grained, low-level motion planning in dynamic environments.

In light of this, we shift our focus to a target-based planning strategy, where the LLM operates at a higher level of abstraction by specifying target destinations rather than explicit movement sequences. This allows us to better evaluate the LLM's potential for high-level decision-making and goal selection.

B. Experiment 2: Target-based with simplified environment

High-level Planning. Building upon the outcomes of Experiment 1, we extended our investigation into the capacity

of LLM agents for high-level planning. The experimental setup remained consistent with Experiment 1, incorporating memory (the preceding 5 steps) and map information. A key modification involved adjusting the structured output to specify the agent's "target position" instead of direct movement directions. This adjustment simplified the task, requiring agents to identify a target based on their understanding of the map, in contrast to movement-based planning which demanded path planning and next-step determination—a more complex task for smaller LLM models.

Impact of Memory on Model Performance. In the experimental results, the Target-based GPT-4o model successfully completed the game within approximately 3 to 4 rounds. Given its impressive performance, we further reduced the model capacity to explore the setting with GPT-4.1-nano. Agents powered by GPT-4.1-nano were unable to complete the game with the original memory context. However, by reducing the memory (removing the history of previous steps and providing only the map information), these agents were able to cooperatively finish the game within 60 rounds. This suggests that for weaker models, extensive memory context can be detrimental, potentially confusing the model. This is because LLMs with limited capacity may struggle to effectively process and integrate a large number of input tokens from extensive memory, diluting their attention and making it difficult to focus on critical real-time information. Additionally, without true persistent memory, they must rely on the provided context, and a long, unstructured history can introduce noise and irrelevant details, impairing planning efficiency. Overall, the target-based planning approach simplified the game, facilitating efficient experimentation with models with limited ability. In the next section, we start to complex the game setting, expecting to see agents more complex cooperation in order to finish the game.

C. Experiment 3: Target-based with complex environment

Building upon the structure of the previous environment, this advanced configuration introduces coal as an additional resource and integrates a tool dependency hierarchy, thereby more accurately emulating the mechanics of realistic open-world survival scenarios. As part of the experimental framework, carefully constructed prompts are submitted to the LLM to generate contextually relevant and goal-oriented responses. Each prompt comprises two principal components: the system prompt and the user input.

The user input provides a concise, objective representation of the current environmental state. It encodes information such as tool-resource dependency chains, agent coordinates, the status of tool construction, and a cumulative summary of collected resources. Notably, to optimize token efficiency and model attention, the input includes only the coordinates of uncollected resources, rather than a full map matrix. This design significantly reduces token overhead and ensures that the model's attention remains concentrated on critical task-relevant features, thereby mitigating the cognitive burden of extraneous spatial data.

The system prompt, in contrast, offers high-level procedural guidance intended to steer agent behavior toward efficient goal

completion. It consists of structured, target-based instructions; exemplar strategies (e.g., for acquiring diamonds); operational constraints; behavioral heuristics; and explicitly defined forbidden actions. This component of the prompt ensures that LLM outputs are not only coherent and grounded but also aligned with domain-specific planning logic and gameplay constraints.

To investigate whether LLMs acquire universal task logic across languages or instead rely on language-specific reasoning patterns, we evaluated our system using prompts in three languages: Chinese, English, and German. The original system prompt was authored in Chinese and then translated into English and German using ChatGPT. Importantly, all translations preserved the original logical constraints and task structure to ensure cross-linguistic comparability.

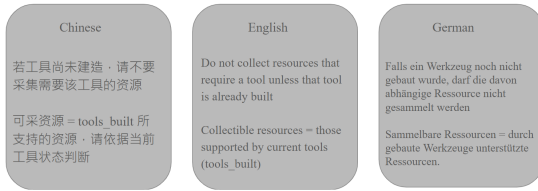


Fig. 12: Multilingual System Prompt Examples

During experimentation, an unexpected pattern emerged. As shown in Figure 13, despite all three prompts encoding the same logic, the performance of the English-language prompt was significantly lower than that of the Chinese and German counterparts. In several trials, the English prompt failed to guide agents to complete key tasks, such as building a furnace and crafting advanced tools. It seems unbelievable, as ChatGPT is heavily trained on English, so English prompt should at least have a performance no worse than other languages. So we decide to explore on this part.

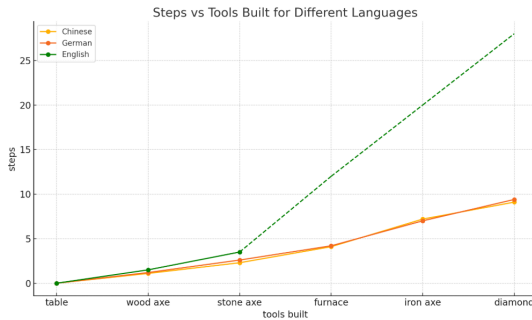


Fig. 13: Comparison about performance of prompts in different languages

To further investigate the observed performance gap, we refined the English prompt while retaining its original logical structure and semantic content. Specifically, we introduced three key modifications: (1) aligning the phrasing more closely with GPT’s native English comprehension patterns to reduce ambiguity; (2) structuring the prompt into clearer semantic blocks; and (3) improving overall linguistic clarity and conciseness.

For example, as shown in Figure 14, the original phrase “Do not move to current position.” was modified to “Agents should not execute move actions that result in staying at their current position.” This change introduces a clear subject (*Agents*), replaces vague phrasing with more formal and descriptive language (*execute move actions*), and clarifies the intended restriction through an explicit consequence clause (*result in staying at their current position*). Such modifications reduce ambiguity, follow common syntactic patterns found in GPT’s training data, and improve the model’s ability to interpret and follow instructions accurately.

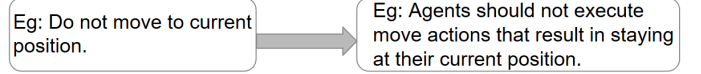


Fig. 14: Example of revision in English prompt

Following these adjustments, we observed a substantial improvement in the performance of the English-language prompt. As shown in Figure 15, the revised prompt achieved task completion rates comparable to those of the Chinese and German versions. This suggests that prompt formulation—particularly in terms of linguistic alignment and structural clarity—plays a critical role in enabling effective reasoning, even in a model extensively trained on English.

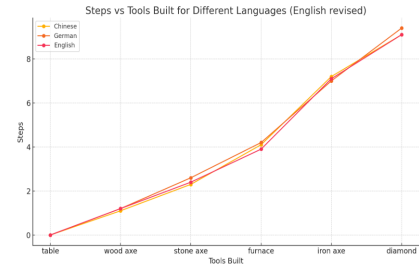


Fig. 15: Performance comparison after the revision of English prompt

Additionally, we investigate on GPT’s response in our original setting, and its performance when the resource counts are increased, and when the memory system is added in this setting. We have some very interesting observations, which are listed in below:

1) Misinterpretation of the user input:

As shown in Figure 16, GPT initially inferred that the table had not been constructed, despite it being successfully created at the beginning of the game, as verified by our debug output. This discrepancy likely stems from GPT’s reliance on textual cues, which can lead to inference errors when explicit state representations are absent or insufficiently emphasized. Fortunately, because the prompt included the explicit `tools_built` status, the model ultimately corrected its initial assumption and proceeded with constructing the furnace.

This observation highlights a key limitation of LLM-based reasoning: without well-structured, context-grounded input, models may fail to retain or appropri-

ately interpret previously completed actions. To mitigate such inconsistencies, it is critical to incorporate clear, structured representations of environment state into the prompt. Doing so improves the model’s ability to ground its decisions in accurate game state information, thereby enhancing consistency and robustness in downstream planning behavior.

=== [Step 8] ===

[GPT Original Response]

Based on the provided information, the next tool to build is the furnace. However, the tool recommendation indicates that the furnace is "not_ready" because a table is missing. Since the table is already built (as per the 'tools_built' status), this seems to be an inconsistency. Assuming the table is available, we can proceed to build the furnace.

Fig. 16: GPT failed to interpret user input correctly

2) Sign of collaboration:

Upon examining GPT’s response, we identify a compelling indication that the language model demonstrates an emergent capacity for inter-agent collaboration and forward-looking reasoning. As illustrated in the region highlighted by the red rectangle in Figure 17, while several agents are assigned immediate goals—such as crafting a stone pickaxe or collecting prerequisite materials—agent_2 autonomously initiates movement toward the diamond tile. Notably, this decision is made despite the current infeasibility of collection, implicitly anticipating the eventual availability of the iron pickaxe, which is required for accessing the target resource.

This behavior suggests that the LLM is capable of modeling not only the present state of the environment but also projecting its potential evolution based on planned tool construction. Such anticipatory behavior indicates a primitive form of decentralized multi-agent planning, wherein agents reason about complementary roles without explicit coordination or communication. These findings raise important questions about the consistency and robustness of such behaviors. In the subsequent sections, we investigate whether this form of implicit coordination emerges consistently across different scenarios, or whether it is merely incidental. Furthermore, we explore prompt engineering techniques and input structures that may enhance the reliability of multi-agent planning within LLM-driven systems.

3) Memory hurts LLM planing:

While memory mechanisms have been shown to enhance LLM performance in previous settings, our experiments indicate that, in this specific task, the inclusion of memory actually degrades performance. In our implementation, we incorporated the last five actions of each agent into the user input as an external memory. However, this often resulted in agents getting stuck in suboptimal states and ultimately failing to complete the task.

```
Here is the plan:
{
  "json": {
    "actions": [
      {
        "agent_id": "agent_1",
        "action": "create",
        "target_tool": "stone pickaxe",
        "reason": "The required materials (1 wood and 1 stone) are available in the shared resource pool."
      },
      {
        "agent_id": "agent_2",
        "action": "move",
        "target_pos": [11, 15],
        "reason": "Move toward the diamond tile to prepare for collection once the iron pickaxe is built."
      },
      {
        "agent_id": "agent_3",
        "action": "move",
        "target_pos": [15, 6],
        "reason": "Move toward the iron tile to collect iron for the iron pickaxe."
      },
      {
        "agent_id": "agent_4",
        "action": "move",
        "target_pos": [1, 13],
        "reason": "Move toward the coal tile to collect coal for the iron pickaxe."
      }
    ]
  }
}
```

Fig. 17: Evidence of Inter-Agent Collaboration in GPT-Generated Plans

Upon further investigation, we identified a number of factors that may explain this counterintuitive behavior. First, the current environment is governed by a target-driven planning mechanism, where each agent’s action directly modifies the shared resource pool and tool construction state. As a result, the environment can be modeled as a first-order partially observable Markov decision process (POMDP), in which the optimal action at each step depends primarily on the current state rather than a longer history of prior actions.

Moreover, we hypothesize several key reasons why adding memory may impair the reasoning ability of the LLM in this context:

- a) **Dilution of attention:** Transformer-based LLMs rely on attention mechanisms to prioritize relevant tokens within the input. When memory is appended as additional context, the number of input tokens increases, thereby diluting attention over a larger context window and potentially drawing focus away from the most critical, real-time state information.
- b) **Lack of true memory integration:** LLMs do not possess persistent internal memory across turns; instead, memory is provided as part of the prompt and must be explicitly reasoned over. Without proper summarization or structuring, LLMs may overlook essential historical dependencies embedded within a long list of prior actions.
- c) **Reduced planning efficiency with long contexts:** Prior research has shown that LLMs tend to underperform when relevant information is located in the middle of a long prompt. Liu et al. [23] observe that models often attend disproportionately to the beginning and end of inputs, a phenomenon known as the "lost in the middle" effect. Additionally, Wu et al. [24] highlight that unfiltered long-term memory access can introduce noise and irrelevant details, suggesting that precise and focused short-term information is more effective for planning tasks.

These findings collectively suggest that, rather than indiscriminately increasing the amount of historical information in the prompt, it is more effective to carefully curate memory content to ensure clarity, relevance, and alignment with the model’s attention capacity.

VII. FINAL THOUGHTS & FUTURE WORKS

A. Final thoughts:

Here are some final thoughts. We, as humans, are naturally good at planning tasks within environments such as those presented in this work, largely due to our prior experience with similar games like Minecraft and intuitive understandings of the physical world, for example, we have a sense of direction. However, LLMs such as ChatGPT, DeepSeek, or Grok likely lack explicit experience or dataset exposure that directly simulates interactive, spatially-structured tasks. This discrepancy raises important questions about the limitations of current LLM architectures when applied to domains requiring precise spatial reasoning and collaborative coordination.

To address this gap, domain adaptation strategies, including reinforcement learning, fine-tuning, and Bayesian learning could potentially equip LLMs with a more structured and robust understanding of physical and spatial interactions. It may be possible to conduct knowledge distillation with a well-trained agent.

However, it remains unclear whether simply accumulating knowledge or memory inherently translates into enhanced LLM performance, as irrelevant or noisy memory can even degrade decision-making quality. Thus, establishing a structured, interpretable, and actionable memory representation/knowledge graph that effectively guides reasoning is essential.

Finally, while LLMs offer valuable capabilities, their limitations highlight that they may not be sufficient as standalone decision-making modules in intelligent agents. Rather, they might better serve as integral components within broader architectures, complemented by specialized modules explicitly designed for spatial reasoning, real-time interaction, and adaptive learning.

B. Future work:

For future work, we plan to enhance the current framework along several dimensions to enable more efficient, scalable, and generalizable multi-agent planning using LLMs.

First, we aim to incorporate a **Memory System** and **Memory Graph** into the environment, enabling the model to learn from past experiences across training epochs. By leveraging historical lessons, we expect the model to generate solutions that are not only correct but also increasingly cost-effective over time. However, the current environment is relatively simple and lacks the complexity required to provide sufficient data and sequential dependencies for effective memory-based learning.

To address this, we propose to significantly **increase environment complexity**. Possible extensions include introducing dynamic obstacles such as lava, walls, and other hazards; implementing a **health system** (e.g., health, thirst, hunger) for each agent; and shifting to a **partially observable** setting

where agents have limited local views of the environment. Additionally, instead of storing collected resources in a shared warehouse, each agent would maintain its own inventory, further complicating resource management.

This more challenging setting would allow us to evaluate whether LLMs can still generate **reasonable and efficient plans** under stricter environmental and operational constraints.

Moreover, we seek to **explicitly study the emergence of inter-agent collaboration**. By assigning distinct specializations to different agents—such as faster movement, enhanced resource collection, or improved crafting skills—we can investigate whether LLMs naturally develop division-of-labor strategies similar to those observed in human societies.

While these extensions focus on testing the internal collaboration and reasoning capabilities of LLMs, we also recognize the importance of **prompt design**. We intend to explore various prompting strategies to better guide LLMs toward generating optimal multi-agent plans. In addition, we plan to integrate **reinforcement learning-based methods** to summarize key lessons from previous epochs. This summary information could be appended to the input prompts in a concise, relevant manner, informing the LLM of potential improvements without introducing conflicts with the real-time environmental state.

During preliminary experiments, we observed that prompts crafted for specific tasks tend to be long and highly specialized, making them difficult to generalize to new application domains. To overcome this, we envision building an **automated prompt generation interface**, where users provide a general description of the environment and the system automatically generates specific, structured, and task-aligned prompts. Furthermore, prompt quality could be iteratively improved during training, leveraging feedback from previous failures and successes.

Collectively, these future directions aim to make LLM-driven multi-agent planning more adaptive, efficient, and widely applicable across diverse and increasingly complex domains.

VIII. CONCLUSIONS

In this project, we developed a new multi-agent simulation environment powered by LLMs, supporting both movement-based and target-based planning strategies. We conducted a series of ablation studies, investigating the impact of prompt optimization, memory integration, and the distinction between joint and individual planning.

By systematically increasing the complexity of the task scenarios, we observed critical findings: effective prompt design becomes increasingly important as task complexity rises, and excessive or irrelevant prompt information severely impairs LLM decision-making. Furthermore, cross-linguistic experiments revealed that prompts must conform to the natural logic and idiomatic structure of the target language; direct translations often degrade performance significantly.

Overall, our results indicate that current LLMs struggle to generalize to complex, spatially grounded tasks. The necessity of frequent manual prompt adjustments for each new environment suggests that LLMs alone are not yet sufficient to serve

as fully autonomous agents in real-world physical settings. There is still a long way to go to reach a final ideal intelligent agent and many other fields that need to be explored.

IX. TEAM MEMBER CONTRIBUTIONS:

Yixiang Wang:

- Built a new simulation framework from scratch
- Conducted multiple movement-based experiments
- Create standard and structured template for interactions between LLM and game environment

Haoran Zhang:

- Proposed the initial ideas of this project and communicated with one ECE PhD
- Implemented Target-based approach
- Conducted multiple target-based experiments

Xianfei Yang:

- Further improve the simulation framework by adding more task to the workflow
- Conducted multiple target-based experiments in upgraded framework
- Conducted optimization experiment on prompt engineering

X. ACKNOWLEDGMENT

REFERENCES

- [1] M. C. De Gennaro and A. Jadbabaie, "Decentralized control of connectivity for multi-agent systems," in *Proceedings of the 45th IEEE Conference on Decision and Control*, pp. 3628–3633, IEEE, 2006.
- [2] Z. H. Ismail, N. Sariff, and E. G. Hurtado, "A survey and analysis of cooperative multi-agent robot systems: challenges and directions," *Applications of Mobile Robots*, vol. 5, pp. 8–14, 2018.
- [3] J. Dinneweth, A. Boubezoul, R. Mandiau, and S. Espié, "Multi-agent reinforcement learning for autonomous vehicles: A survey," *Autonomous Intelligent Systems*, vol. 2, no. 1, p. 27, 2022.
- [4] H. Zhang and H. Chen, "Efficient 3d transformer with cluster-based domain-adversarial learning for 3d medical image segmentation," in *2023 IEEE 20th International Symposium on Biomedical Imaging (ISBI)*, pp. 1–5, IEEE, 2023.
- [5] A. Camarillo, J. Ríos, and K.-D. Althoff, "Knowledge-based multi-agent system for manufacturing problem solving process in production plants," *Journal of manufacturing systems*, vol. 47, pp. 115–127, 2018.
- [6] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat, *et al.*, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.
- [7] L. Kraemer and B. Banerjee, "Multi-agent reinforcement learning as a rehearsal for decentralized planning," *Neurocomputing*, vol. 190, pp. 82–94, 2016.
- [8] M. Bettini, A. Prorok, and V. Moens, "Benchmarl: Benchmarking multi-agent reinforcement learning," *Journal of Machine Learning Research*, vol. 25, no. 217, pp. 1–10, 2024.
- [9] H. Yang, J. Chen, M. Siew, T. Llorido-Botran, and C. Joe-Wong, "Llm-powered decentralized generative agents with adaptive hierarchical knowledge graph for cooperative planning," *arXiv preprint arXiv:2502.05453*, 2025.
- [10] D. Hafner, "Benchmarking the spectrum of agent capabilities," *arXiv preprint arXiv:2109.06780*, 2021.
- [11] K. Zhang, Z. Yang, and T. Basar, "Networked multi-agent reinforcement learning in continuous spaces," in *2018 IEEE conference on decision and control (CDC)*, pp. 2771–2776, IEEE, 2018.
- [12] J. Chen, T. Lan, and C. Joe-Wong, "Rgmcomm: Return gap minimization via discrete communications in multi-agent reinforcement learning," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, pp. 17327–17336, 2024.
- [13] S. Karten, M. Tucker, S. Kailas, and K. Sycara, "Towards true lossless sparse communication in multi-agent systems," in *2023 IEEE international conference on robotics and automation (icra)*, pp. 7191–7197, IEEE, 2023.
- [14] H. Zhang, W. Du, J. Shan, Q. Zhou, Y. Du, J. B. Tenenbaum, T. Shu, and C. Gan, "Building cooperative embodied agents modularly with large language models," *arXiv preprint arXiv:2307.02485*, 2023.
- [15] Y. Wu, S. Y. Min, S. Prabhume, Y. Bisk, R. R. Salakhutdinov, A. Azaria, T. M. Mitchell, and Y. Li, "Spring: Studying papers and reasoning to play games," *Advances in Neural Information Processing Systems*, vol. 36, pp. 22383–22687, 2023.
- [16] J. Park, B. Min, K. Son, J. Y. Song, X. Ma, and J. Kim, "Choicemates: Supporting unfamiliar online decision-making with multi-agent conversational interactions," *arXiv preprint arXiv:2310.01331*, 2023.
- [17] M. Li, S. Zhao, Q. Wang, K. Wang, Y. Zhou, S. Srivastava, C. Gokmen, T. Lee, E. L. Li, R. Zhang, *et al.*, "Embodied agent interface: Benchmarking llms for embodied decision making," *Advances in Neural Information Processing Systems*, vol. 37, pp. 100428–100534, 2024.
- [18] S. Bubeck, V. Chadrsekaran, R. Eldan, J. Gehrke, E. Horvitz, E. Kamar, P. Lee, Y. T. Lee, Y. Li, S. Lundberg, *et al.*, "Sparks of artificial general intelligence: Early experiments with gpt-4," 2023.
- [19] C. Irugalbandara, "Meaning typed prompting: A technique for efficient, reliable structured output generation," *arXiv preprint arXiv:2410.18146*, 2024.
- [20] S. Yang, O. Nachum, Y. Du, J. Wei, P. Abbeel, and D. Schuurmans, "Foundation models for decision making: Problems, methods, and opportunities," *arXiv preprint arXiv:2303.04129*, 2023.
- [21] P. Anokhin, N. Semenov, A. Sorokin, D. Evseev, M. Burtsev, and E. Burnaev, "Arigraph: Learning knowledge graph world models with episodic memory for llm agents," *arXiv preprint arXiv:2407.04363*, 2024.
- [22] L. Fan, G. Wang, Y. Jiang, A. Mandlekar, Y. Yang, H. Zhu, A. Tang, D.-A. Huang, Y. Zhu, and A. Anandkumar, "Minedojo: Building open-ended embodied agents with internet-scale knowledge," *Advances in Neural Information Processing Systems*, vol. 35, pp. 18343–18362, 2022.
- [23] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *arXiv preprint arXiv:2307.03172*, 2023.
- [24] Y. Wu, M. N. Rabe, D. Hutchins, and C. Szegedy, "Memorizing transformers," 2022.